19
Reflection
e informazioni run-time sui tipi

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Illustrare il concetto di run-time type information
- Mostrare le principali funzionalità della Reflection
- Descrivere il meccanismo delle annotazioni

Argomenti

- Oggetti `Class` e loro uso
- Reflection API
- Annotazioni di Java
- Testing con `JUnit`

Outline

1. Classi, caricamento e JVM
2. Reflection API
3. Un esempio di applicazione
4. Annotazioni
5. JUnit

Il classfile

Classi e JVM

- Ogni classe Java (interfaccia, enumerazione, inner o outer) produce un file `.class` ad opera del compilatore
- Nel caso di classi "inner" il nome di tale `.class` è `<outer>$<inner>`, nel caso di anonima è `<outer>$<numero>`
- Tale `.class` è disponibile nel folder di uscita e innestato a seconda del suo package
- Il contenuto informativo del `.class` è desunto (per compilazione) da quello del `.java`, solo espresso in un linguaggio diverso che garantisce la correttezza del contenuto e le performance di chi deve leggerlo

È possibile ispezionare il contenuto di un `.class`

- Esempio di comando: `javap -v Counter.class`
- Comando in modalità verbose

Classe Counter

```
class Counter{

    /* Il campo è reso inaccessibile direttamente */
    private int countValue;

    /* E' il costruttore che inizializza i campi */
    public Counter(){
      this.countValue=0;
    }

    /* Unico modo per osservare lo stato */
    public int getValue(){
      return this.countValue;
    }

    /* Unico modo per modificare lo stato */
    public void increment(){
      this.countValue++;
    }
}
```

Contenuto del classfile (1/2)

```
Classfile .../Counter.class
  Last modified Sep 20, 2018; size 361 bytes
  MD5 checksum 2c941be1d3cfd60de02143767ce146cc
  Compiled from "Counter.java"
class Counter
  SourceFile: "Counter.java"
  minor version: 0
  major version: 51
  flags: ACC_SUPER
Constant pool:
   #1 = Methodref          #4.#16         //  java/lang/Object."<init>":()V
   #2 = Fieldref           #3.#17         //  Counter.countValue:I
   #3 = Class              #18            //  Counter
   #4 = Class              #19            //  java/lang/Object
   #5 = Utf8               countValue
   #6 = Utf8               I
   #7 = Utf8               <init>
   #8 = Utf8               ()V
   #9 = Utf8               Code
  #10 = Utf8               LineNumberTable
  #11 = Utf8               getValue
  #12 = Utf8               ()I
  #13 = Utf8               increment
  #14 = Utf8               SourceFile
  #15 = Utf8               Counter.java
  #16 = NameAndType        #7:#8          //  "<init>":()V
  #17 = NameAndType        #5:#6          //  countValue:I
  #18 = Utf8               Counter
  #19 = Utf8               java/lang/Object
```

Contenuto del classfile (2/2)

```
{  public Counter();
    flags: ACC_PUBLIC   Code:
      stack=2, locals=1, args_size=1
         0: aload_0
         1: invokespecial #1                  // Method java/lang/Object."<init>":()V
         4: aload_0
         5: iconst_0
         6: putfield      #2                  // Field countValue:I
         9: return
      LineNumberTable:        line 7: 0 line 8: 4        line 9: 9

  public int getValue();
    flags: ACC_PUBLIC Code:
      stack=1, locals=1, args_size=1
         0: aload_0
         1: getfield      #2                  // Field countValue:I
         4: ireturn
      LineNumberTable:    line 13: 0
  public void increment();
    flags: ACC_PUBLIC Code:
      stack=3, locals=1, args_size=1
         0: aload_0
         1: dup
         2: getfield      #2                  // Field countValue:I
         5: iconst_1
         6: iadd
         7: putfield      #2                  // Field countValue:I
        10: return
      LineNumberTable:          line 18: 0        line 19: 10   }
```

Caricamento delle classi e la JVM (1/2)

Caricamento: chi?

- La JVM è un programma solitamente scritto in C/C++
- HotSpot di OpenJDK (> 250K linee): `http://hg.openjdk.java.net/jdk8/jdk8/hotspot/`
- Dispone di uno o più class-loader (realizzabili anche in Java dall'utente, estendendo `java.lang.ClassLoader`)
- Hanno il compito di cercare i classfile che servono, e di "caricarli" nella JVM

Caricamento: quando?

- Tale caricamento **non** avviene necessariamente all'avvio: ogni classe viene caricata al momento del suo primo utilizzo!! (Schema by-need)
- Alla prima `new`, o chiamata statica, o se serve una sottoclasse!
- (o con una richiesta esplicita come vedremo)

Caricamento delle classi e la JVM (2/2)

Caricamento: da dove?

- Dal file system, attraverso il classpath e navigando i package
- Eventualmente dentro ai file JAR
- In alcune modalità, può caricare le classi anche via rete!

Caricamento: cosa succede?

- La JVM prepara una opportuna struttura dati in memoria
- Inizializza i campi statici (e chiamando l'inizializzatore statico se definito)

Nota: esiste anche l'inizializzatore "non statico"

- viene richiamato all'atto della creazione dell'oggetto

Una parentesi: gli initializer

Static initializer

- sintassi `static{...}`
- eseguito "subito", prima di qualunque accesso alla classe
- in genere inizializza campi statici
- potrebbe eseguire altro codice di inizializzazione

Non-static initializer

- sintassi `{...}`
- eseguito prima di chiamare il costruttore
- in genere inizializza campi istanza
- potrebbe eseguire altro codice di inizializzazione

Inizializzatori all'opera

```java
public class Initializers {

  static { // static initializer.. è eseguito "subito"
    System.out.println("Static initializer executed...");
    SET = Set.of(10,20,30); // ad esempio inizializza i campi statici
  }

  { // non-static initializer.. è eseguito prima del costruttore
    System.out.println("Non-static initializer executed...");
    set = Set.of(10,20,30,40);
  }

  Initializers(){
    System.out.println("Constructor executed...");
  }

  private static final Set<Integer> SET;
  private final Set<Integer> set;
  // use of non-static initializers with anonymous classes
  private final Set<Integer> set2 = new HashSet<>() {{ add(1); add(2); }};

  public static void main(String[] args) {
    System.out.println("Let's create an object..");
    Initializers i = new Initializers();
    System.out.println(i.set2);
  }

}
```

Ispezioniamo la dinamica di caricamento (e istanziazione)

```
class A{
  static { System.out.println("A.class caricato");}
  { System.out.println("A.class istanziato"); }
  { System.out.println("A.class istanziato"); }
}

class B extends A{
  static { System.out.println("B.class caricato");}
  { System.out.println("B.class istanziato"); }
}

public class Loading {
  static { System.out.println("Loading.class caricato");}

  public static void main(String[] args) {
    System.out.println("main partito");
    new B();
    System.out.println("creato un oggetto di B");
  }
  /*  Loading.class caricato
    main partito
    A.class caricato
    B.class caricato
    A.class istanziato
    A.class istanziato
    B.class istanziato
    creato un oggetto di B  */
}
```

Perché questa gestione "by-need" del caricamento?

Alcune motivazioni

- Permette alle applicazioni di "partire" più velocemente, in quanto non si carica tutto, solo quello che serve mano a mano
- In scenari di rete, consente di dover caricare da remoto solo sottoparti di applicazioni
- In scenari avanzati si potrebbero anche "scaricare" (togliere dalla JVM) le classi che sembrano non servire più, o addirittura fare "hot-swapping" (modifica di una classe)
- Alcune classi potrebbe essere aggiunte al volo per aumentare funzionalità senza spegnere l'applicazione
- È possibile ispezionare e usare il contenuto delle classi via reflection

Outline

1. Classi, caricamento e JVM
2. Reflection API
3. Un esempio di applicazione
4. Annotazioni
5. JUnit

Reflection

Packages `java.lang` e `java.lang.reflect`

Forniscono una libreria che interagisce con la JVM per..

- dare una rappresentazione "ad oggetti" del contenuto di una classe
- direttamente istanziare un oggetto, invocare metodi, accedere a campi
- forzare il caricamento di una classe

Sulla modalità "via reflection"

È più flessibile ma..

- è più lenta, anche di 1-2 ordini di grandezza (ma ottimizzabile)
- non interagisce con i controlli del compilatore (genera eccezioni..)
- pone problemi di security (che non analizzeremo in dettaglio..)
- consente di programmare accessi a classi non note a priori

$\Rightarrow$ va usata di conseguenza.. quindi non abusarne

Motivazioni

Tecniche messe a disposizione

- Unire classi non note ad una applicazione durante il suo funzionamento
- Trattare una stringa come identificatore del linguaggio
- Interagire con oggetti in modo dnamico, bypassando i test del compilatore

Applicazioni della reflection

- Estendibilità: Si può fare uso di classi esterne create/caricate "al volo", per modificare dinamicamente il comportamento di una applicazione
- Ambienti di sviluppo: Poter ispezionare la struttura di una classe o libreria
- Framework di Java: annotazioni, serializzazione, dynamic proxies,...

La classe `java.lang.Class`

Ogni suo oggetto rappresenta un tipo disponibile nella JVM

- una classe (outer o inner, astratta o non), una interfaccia, una enumerazione, un array, i tipi primitivi e anche void
- non i tipi generici (`ArrayList<String>`) ma le classi generiche si

Come si ottiene un oggetto di `Class`? In tre modi:

- `String.class`
- `new String("this is a string").getClass()`
- `Class.forName("java.lang.String")`

Genericità di `Class`

- Solo via `String.class` si può ottenere un oggetto di tipo `Class<String>`, che avrà quindi metodi che "sanno" di lavorare su stringhe (ad esempio per creare oggetti di tipo stringa)
- Altrimeni restituisce un `Class<?>`, e quindi prima o poi serviranno delle conversioni (con possibili cast "unchecked")

Esempi

```java
public class TryClass {

  public static void main(String[] args) throws Exception {

    // Unico caso di recupero del corretto tipo generico
    final Class<String> c = String.class;
    System.out.println(c);
    System.out.println("oggetto "+c.newInstance());

    // Si può ottenere una class da una stringa calcolata
    final Class<?> c2 = Class.forName("java.lang" + ".String");
    System.out.println("oggetto "+(String)c2.newInstance());

    // Accesso alla classe di un oggetto
    final Object o = "3";
    final Class<?> c3 = o.getClass();
    System.out.println(c3); //String

    // Cast "unchecked" per recuperare il generico
    final Class<Integer> c4 = (Class<Integer>)o.getClass();
    System.out.println(c4);

    // Cast "unchecked" per recuperare il generico
    String s = cloneObject("prova");
    java.util.Date d = cloneObject(new java.util.Date());
    System.out.println(s.length()+" "+d);
  }

  private static <T> T cloneObject(T t) throws Exception {
    return (T)t.getClass().newInstance();
  }
}
```

Esempi

```java
public class UseClass {
  public static void main(String[] args) throws ClassNotFoundException {
    final Class<String> c = String.class;
    System.out.println(c.getName()+" "+c.getCanonicalName());
    //java.lang.String java.lang.String
    final Class<Integer> ci = (Class<Integer>)new Integer(5).getClass();
    System.out.println(ci.getName()+" "+ci.getCanonicalName());
    //java.lang.Integer java.lang.Integer
    final Class<?> ca = new int[20].getClass();
    System.out.println(ca.getName()+" "+ca.getCanonicalName());
    //[I int[]
    final Class<?> cint = ca.getComponentType();
    System.out.println(cint.getName()+" "+cint.getCanonicalName());
    //int int
    final Class<?> cl = Class.forName("java.util.List");
    System.out.println(cl.getName()+" "+cl.getCanonicalName());
    //java.util.List java.util.List
    final Class<?> can = new Object(){
      public String toString(){ return "none";}
    }.getClass();
    System.out.println(can.getName()+" "+can.getCanonicalName());
    //it.unibo.apice.oop.p19reflection.UseClass$1 null
  }
}
```

Alcuni metodi di java.lang.Class

```java
public final class Class<T> implements ...  {

    public static Class<?> forName(String className) throws ClassNotFoundException {...}

    public boolean isInterface();
    public boolean isArray();
    public boolean isPrimitive();
    public boolean isAnonymousClass() {...}
    public boolean isLocalClass() {...}
    public boolean isMemberClass() {...}
    public boolean isEnum() {...}

    public T[] getEnumConstants() {...}
    public Class<?> getComponentType();
    public T cast(Object obj) {...}

    public T newInstance() throws ...{...}

    // Accessing the structure.. can throw SecurityException
    public Field[] getFields() throws ... {...}
    public Method[] getMethods() throws ... {...}
    public Constructor<?>[] getConstructors() throws ... {...}

    public Field getField(String name) throws NoSuchFieldException, .. {...}
    public Method getMethod(String name, Class<?>... parameterTypes)
        throws NoSuchMethodException, .. {...}
    public Constructor<T> getConstructor(Class<?>... parameterTypes)
        throws NoSuchMethodException, .. {...}

    public Field[] getDeclaredFields() throws SecurityException {...}
    ... // All versions with 'Declared'
}
```

Alcuni metodi di java.lang.reflect.Field

```
public final class Field extends ... {

    public Class<?> getDeclaringClass() {...}
    public String getName() {...}
    public int getModifiers() {...}
    public boolean isEnumConstant() {...}
    public Class<?> getType() {...}

    public Object get(Object obj)
        throws IllegalArgumentException, IllegalAccessException{...}
    public boolean getBoolean(Object obj)
        throws IllegalArgumentException, IllegalAccessException{...}
    public byte getByte(Object obj)
        throws IllegalArgumentException, IllegalAccessException{...}
    ...

    public void set(Object obj, Object value)
        throws IllegalArgumentException, IllegalAccessException{...}
    ...
}
```

Alcuni metodi di java.lang.reflect.Constructor

```
public final class Constructor<T> extends ... {

    public Class<T> getDeclaringClass() {...}
    public String getName() {...}
    public int getModifiers() {...}
    public Class<?>[] getParameterTypes() {...}
    public Class<?>[] getExceptionTypes() {...}

    public T newInstance(Object ... initargs)
        throws InstantiationException, IllegalAccessException,
               IllegalArgumentException, InvocationTargetException {...}

    public boolean isVarArgs() {...}
    ...
}
```

Alcuni metodi di java.lang.reflect.Method

```
public final class Method extends ... {

    public Class<?> getDeclaringClass() {...}
    public String getName() {...}
    public int getModifiers() {...}
    public Class<?> getReturnType() {...}
    public Class<?>[] getParameterTypes() {...}
    public Class<?>[] getExceptionTypes() {...}
    public boolean isVarArgs() {...}

    public Object invoke(Object obj, Object... args)
        throws IllegalAccessException, IllegalArgumentException,
           InvocationTargetException {...}
}
```

Esempio di chiamata dinamica di metodo

```
1  public class UseReflection {
2
3    private static Object callGetter(Object receiver, String getterName)
       throws Exception {
4      return receiver.getClass().getMethod(getterName).invoke(receiver);
5    }
6
7    public static void main(String[] args) throws Exception {
8      System.out.println(callGetter("prova","isEmpty"));
9      System.out.println(callGetter(new java.util.Date(),"getDate"));
10     System.out.println(callGetter(new Object(),"hashCode"));
11
12     class A{
13       public void createDialog() {
14         javax.swing.JOptionPane.showMessageDialog(null, "It worked!");
15       }
16     }
17     System.out.println(callGetter(new A(),"createDialog"));
18     //System.out.println(callGetter(new A(),"createDial"));
19   }
20 }
```

Reflection e esecuzione "dinamica"

Applicazioni dinamiche

- Sono applicazioni che riescono ad eseguire codice aggiunto dinamicamente dopo che l'applicazione stessa è partita
- Questo tipo di meccanismo è molto importante in sistemi che non è possibile "spegnere"

Tecnica

- Un gestore dell'applicazione compila nuove classi e le aggiunge al CLASSPATH
- L'applicazione è già stata costruita in modo da accedere a certe classi da eseguire tramite il loro nome, eseguendone certi metodi

Caricamento ed esecuzione automatica

```java
public class DynamicExecution {

  private static final String Q_CLASS = "Insert fully-qualified class name: ";
  private static final String Q_METH = "Insert name of method to call: ";
  private static final String L_OK = "Everything was ok! The result is..";
  private static final String E_RET = "Wrong return type";

  public static void main(String[] s) throws Exception {
    BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
    while (true) {
      System.out.println(Q_CLASS);
      final String className = reader.readLine();
      System.out.println(Q_METH);
      final String methName = reader.readLine();
      final Class<?> cl = Class.forName(className); // Ottiene la classe
      final Constructor<?> cns = cl.getConstructor(); // Ottiene il costruttore
      final Method met = cl.getMethod(methName);      // Ottiene il metodo
      if (!met.getReturnType().isAssignableFrom(String.class)) {
        throw new NoSuchMethodException(E_RET);   // Il metodo deve tornare String
      }
      final Object o = cns.newInstance();             // Istanzia l'oggetto
      final String result = (String) met.invoke(o);   // Chiama il metodo
      System.out.println(L_OK);
      System.out.println(result);
      System.out.println();
    }
  }
}
```

Outline

1. Classi, caricamento e JVM
2. Reflection API
3. Un esempio di applicazione
4. Annotazioni
5. JUnit

Sviluppiamo un semplice esempio

Realizzazione automatizzata del `toString` – un mero esempio

- Scrivere i `toString` è piuttosto noioso e ripetitivo
- Alcuni IDE (come Eclipse) lo generano automaticamente
- Come potremmo fornire un supporto programmato?
- Il principale problema è indicare dinamicamente, di volta in volta, come produrre la stringa sulla base delle proprietà di interesse
- Forniamo una soluzione base, facilmente estendibile dallo studente

Idea

- Fornisco un metodo statico in una classe di funzionalità varie
- Accetta l'oggetto da stampare e una descrizione di cosa stampare
- Esempio: il nome della proprietà da ritrovare (assumendo ci sia un getter)
- Ritorna la stringa creata

objectToString

```java
public class PrintObjectsUtilities {

  public static String objectToString(Object o, String... getters) {
    try {
      String out = o.getClass().getSimpleName()+": ";
      for (String getter : getters) {
        // Sistemo la maiuscola iniziale
        getter = getter.substring(0,1).toUpperCase() + getter.substring(1);
        // Trovo il getter e lo invoco
        final Method m = o.getClass().getMethod("get" + getter);
        final Object res = m.invoke(o);
        // Aggiungo la stringa
        out += " " + getter + " -> ";
        out += res.getClass().isArray()
              ? Arrays.deepToString((Object[])res)
              : res.toString();
        out += " |";
      }
      return out.substring(0,out.length()-2);
    } catch (Exception e) {
      return null;
    }
  }
}
```

Classi di prova

```java
public class Person {
    ...

  public Person(String name, int id) {..}

  public String getName() {..}

  public int getId() {..}

}
```

```java
public class Teacher extends Person {
    ...

  public Teacher(String name, int id, String... courses) {..}

  public String[] getCourses() {..}
}
```

Uso di objectToString

```java
import static it.unibo.apice.oop.p19reflection.classes.PrintObjectsUtilities
    .*;

public class UsePrintObjectUtilities {

  public static void main(String[] args) {
    Person p = new Person("Mario",101);
    Teacher t = new Teacher("Gino",102,"PC","OOP");
    System.out.println(objectToString(p));
    System.out.println(objectToString(p,"name"));
    System.out.println(objectToString(p,"name","id"));
    System.out.println(objectToString(t,"name","id","courses"));

    Counter c = new Counter();
    c.increment();
    c.increment();
    System.out.println(objectToString(c,"value"));

  }

}
```

```
Person
Person:  Name -> Mario
Person:  Name -> Mario | Id -> 101
Teacher:  Name -> Gino | Id -> 102 | Courses -> [PC, OOP]
```

Outline

1. Classi, caricamento e JVM
2. Reflection API
3. Un esempio di applicazione
4. Annotazioni
5. JUnit

Il Java annotation framework

Annotazioni in Java

- Sono un meccanismo usato per "annotare" pezzi di codice
- Il compilatore di default ignora queste annotazioni
- A run-time, via reflection, è possibile verificare quali annotazioni e dove sono presenti
- È anche possibile istruire il compilatore a rigettare annotazioni mal formate
- Java fornisce alcune annotazioni standard

Motivazioni

- Rendere il linguaggio più flessibile
- Consentire di realizzare piccole aggiunte "programmate" al linguaggio

Un primo esempio di annotazione: Override

```
class MyClass extends SuperClass{
    ...
    @Override
    public void myMethod(...){...}
```

Elementi principali

- Abbiamo annotato con `@Override` il metodo `myMethod`
- `javac` è istruito a rigettare questo codice se `SuperClass` non definisce `myMethod`
  - stessa cosa quando si implementa il metodo di una interfaccia
- Serve a evitare errori di nome nell'indicazione di `myMethod`
- È prassi usarli sempre quando si fa overriding
- Eclipse li aggiunge e segnala eventuali errori

Esempio @Override

```
1 public class A {
2   void metodo(int x){}
3 }
4
5 class B extends A{
6   @Override
7   void metod(int x){} // Il compilatore segnala errore
8 }
```

Altre annotazioni di libreria

Cosa sono le annotazioni?

- Sono indicabili come sorta di interfacce
- Ogni package può esporre le proprie
- Le librerie di Java ne espongono varie

`java.lang.Override`

- Controllo statico del corretto overriding

`java.lang.SuppressWarnings`

- Dichiara del codice essere corretto: non genererà warning!
- Vuole come parametro il warning da disabilitare

Altri

- `java.lang.Deprecated`: marca "deprecato" il metodo
- `java.lang.FunctionalInterface`: verifica che un solo metodo richieda implementazione
- Varie definite nel package `java.lang.annotation`

Esempio @SuppressWarnings

```java
public class Vector<X>{
    private Object[] elements = new Object[10]; // Deposito elementi
    private int size = 0;   // Numero di elementi

    public void addElement(X e){
      if (this.size == elements.length){
          this.expand();  // Se non c'è spazio
      }
      this.elements[this.size] = e;
      this.size++;
    }

    @SuppressWarnings("unchecked")
    public X getElementAt(int position){
      return (X)this.elements[position];
    }

    public int getLength(){
      return this.size;
    }

    private void expand(){  // Raddoppio lo spazio..
      Object[] newElements = new Object[this.elements.length*2];
      for (int i=0; i < this.elements.length; i++){
          newElements[i] = this.elements[i];
      }
      this.elements = newElements;
    }
```

Annotazioni custom

Definire le proprie annotazioni

- È possibile definire le proprie annotazioni, con una sintassi che ricalca molto da vicino quella delle interfacce
- È possibile via reflection capire quali metodi/campi/classi/costruttori sono stati annotati

Dove si possono inserire?

Per annotare la dichiarazione di classi, campi, metodi e costruttori
Non è al momento consentito annotare anche i tipi mentre li si usano

Come li si dichiara

- L'uso di una annotazione genera un oggetto ispezionabile
- L'annotazione dichiara l'interfaccia di tale oggetto, e come lo si deve inizializzare
- Si forniscono anche informazioni ulteriori

Un esempio di applicazione

Riprendiamo l'esempio `objectToString()`

- Costruiamo una gestione delle annotazioni che permetta di annotare alcuni metodi getter
- E costruiamo una nuova `objectToString` che stampi controllando tali annotazioni

Dichiarazione @ToString

```java
package it.unibo.apice.oop.p19reflection.annotations;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Documented // genera documentazione javadoc
@Target(ElementType.METHOD) // potrà essere usato solo nei metodi
@Retention(RetentionPolicy.RUNTIME) // sarà visibile a run-time
public @interface ToString {
    String customName() default ""; // propr. specificabili

    String associationSymbol() default "->";

    String separator() default ",";
}
```

Esempio di come si usa l'annotazione

```
public class Person {

  private final String name;
  private final int id;

  public Person(final String name, final int id) {
    this.name = name;
    this.id = id;
  }

  @ToString
  public String getName() {
    return name;
  }

  @ToString
  public int getId() {
    return id;
  }
}
```

Altro esempio

```java
public class Product {

  private final String name;
  private final int id;
  private final double quantity;

  public Product(final String name, final int id, final double quantity) {
    this.name = name;
    this.id = id;
    this.quantity = quantity;
  }

  @ToString( separator = ";" )
  public String getName() {
    return name;
  }

  @ToString( separator = ";", associationSymbol = ":")
  public int getId() {
    return id;
  }

  @ToString( separator = ";", customName = "qty")
  public double getQuantity() {
    return quantity;
  }
}
```

Metodo objectToString()

```
1  import java.lang.reflect.Method;
2
3  public class PrintObjectsUtilities {
4
5    public static String objectToString(Object o) {
6      try {
7        String out = "";
8        for (final Method m : o.getClass().getMethods()) {
9          if (m.isAnnotationPresent(ToString.class) && m.getParameterTypes().length==0){
10           final ToString annotation = m.getAnnotation(ToString.class);
11           out += annotation.customName().equals("") ? m.getName()
12                                                     : annotation.customName();
13           out += annotation.associationSymbol();
14           out += m.invoke(o) + annotation.separator() + " ";
15         }
16       }
17       return out;
18     } catch (Exception e) {
19       return null;
20     }
21   }
22 }
```

Uso objectToString()

```
public class UsePrintObjectsUtilities {

  public static void main(String[] args) {
    final Person p = new Person("Marco", 100);
    System.out.println(PrintObjectsUtilities.objectToString(p));
    // getName->Marco, getId->100,

    final Product pr = new Product("Pr", 200, 100000);
    System.out.println(PrintObjectsUtilities.objectToString(pr));
    // qty->100000.0; getName->Pr; getId:200;
  }
}
```

Outline

1. Classi, caricamento e JVM
2. Reflection API
3. Un esempio di applicazione
4. Annotazioni
5. JUnit

Il framework per i collaudi software JUnit

Una applicazione delle annotazioni di Java... Idea:

- Creare delle classi di test, con metodi che realizzano degli scenari d'uso di certe classi da testare
- Tali metodi sono propriamente annotati
- Alla fine del loro lavoro tali metodi asseriscono se il risultato atteso è giusto
- Da Eclipse semplice esecuzione della classe come "JUnit test"

Per verificare se e quanti test passano.. da linea di comando

- Compilazione specificando la libreria JUnit
- Esecuzione del framework JUnit

```
javac -cp .:/opt/java/junit-4.11.jar *.java
java -cp .:/opt/java/junit-4.11.jar:\
/opt/java/hamcrest-core-1.3.jar\
org.junit.runner.JUnitCore CounterTest
```

Classe per il collaudo di un contatore

```
import static org.junit.Assert.*;

import org.junit.Test;

public class CounterTest {

  @Test
  public void test1() {
    Counter c = new Counter();
    c.increment();
    c.increment();
    assertTrue("Increment does not work wrt getValue", c.getValue()==2);
  }

  @Test
  public void test2() {
    Counter c = new Counter();
    assertTrue("getValue is not initially zero", c.getValue()==0);
  }


}
```

Altre modalità di asserire il risultato di un test

Si veda: https://github.com/junit-team/junit/wiki/Assertions

```
// many things to import

public class AssertTests {
  @Test
  public void testAssertFalse() {
    org.junit.Assert.assertFalse("failure - should be false", false);
  }

  @Test
  public void testAssertSame() {
    Integer aNumber = Integer.valueOf(768);
    org.junit.Assert.assertSame("should be same", aNumber, aNumber);
  }

  // JUnit Matchers assertThat
  @Test
  public void testAssertThatEveryItemContainsString() {
    org.junit.Assert.assertThat(
        Arrays.asList(new String[] { "fun", "ban", "net" }),
        everyItem(containsString("n")));
  }
  ..
}
```

Testiamo il RangeIterator: e modifichiamo se serve..

```
public class RangeIterator implements java.util.Iterator<Integer>{

    private int current;
    final private int stop;

    public RangeIterator(final int start, final int stop){
      if (start>stop){
        throw new IllegalArgumentException();
      }
      this.current = start;
      this.stop = stop;
    }

    public Integer next(){
      if (!this.hasNext()){  // next() oltre i limiti
          throw new java.util.NoSuchElementException();
      }
      return this.current++;
    }

    public boolean hasNext(){
      return this.current <= this.stop;
    }

    public void remove(){
      throw new UnsupportedOperationException();
    }
}
```

Possibile codice JUnit: 1/2

```java
package it.unibo.apice.oop.p19reflection.annotations;

import static org.junit.Assert.*;
import java.util.*;

import org.junit.Test;

public class TestRangeIterator {

  @Test
  public void testStandardBehaviour() {
    final Iterator<Integer> it = new RangeIterator(5,7);
    assertTrue(it.hasNext());
    assertEquals(5,it.next().intValue());
    assertTrue(it.hasNext());
    assertEquals(6,it.next().intValue());
    assertTrue(it.hasNext());
    assertEquals(7,it.next().intValue());
    assertFalse(it.hasNext());
  }

  @Test
  public void testStartEqualsStop() {
    Iterator<Integer> it = new RangeIterator(5,5);
    assertTrue(it.hasNext());
    assertEquals(5,it.next().intValue());
    assertFalse(it.hasNext());
  }
```

Possibile codice JUnit: 2/2

```java

  @Test
  public void testRemoveRaisesException() {
    Iterator<Integer> it = new RangeIterator(5,10);
    try {
      it.remove();
      fail();
    } catch (UnsupportedOperationException e) {
    }
  }

  // Variante migliorativa
  @Test(expected = UnsupportedOperationException.class)
  public void testRemoveRaisesException2() {
    Iterator<Integer> it = new RangeIterator(5,10);
    it.remove();
  }

  @Test(expected = NoSuchElementException.class)
  public void testNoSuchElement() {
    Iterator<Integer> it = new RangeIterator(5,6);
    it.next();
    it.next();
    it.next();
  }

  @Test(expected = IllegalArgumentException.class)
  public void testIllegalArguments() {
    new RangeIterator(6,4);
  }
}
```

Altro esempio: esercizio Graph

```java
import java.util.*;

public interface Graph<N> {

  // Adds a node: nothing happens if node is null or already there
  void addNode(N node);

  // Adds an edge: nothing happens if source or target are null
  void addEdge(N source, N target);

  // Returns all the nodes
  Set<N> nodeSet();

  // Returns all the nodes directly targeted from node
  Set<N> linkedNodes(N node);

  // Gets one sequence of nodes connecting source to path
  List<N> getPath(N source, N target);

}
```

Un esempio di semplicissimo test

```java
public class TestGraphImpl {

  @Test
  public void testNodeSet() {
    final Graph<String> g = null; //new GraphImpl<>();
    assertEquals(g.nodeSet().size(),0);

    g.addNode("a");
    g.addNode("b");
    g.addNode("c");
    g.addNode("d");
    g.addNode("e");
    assertEquals(g.nodeSet().size(),5);
    assertThat(g.nodeSet(),hasItems("a","b","c","d","e"));
    assertEquals(g.linkedNodes("c").size(),0);

    g.addEdge("a","b");
    g.addEdge("b","c");
    g.addEdge("c","d");
    g.addEdge("d","e");
    g.addEdge("c","a");
    assertEquals(g.linkedNodes("c").size(),2);
    assertThat(g.linkedNodes("c"),hasItems("a","d"));

    assertEquals(g.getPath("b", "a"),Arrays.asList("b","c","a"));
  }

}
```

Variante migliorativa (1/2)

```java
public class TestGraphImplImproved {

    private Graph<String> g;

    @Before     // initalisation code executed prior to each test
    public void initialise() {
        //g = new GraphImpl<>();
        g.addNode("a");
        g.addNode("b");
        g.addNode("c");
        g.addNode("d");
        g.addNode("e");
        g.addEdge("a", "b");
        g.addEdge("b", "c");
        g.addEdge("c", "d");
        g.addEdge("d", "e");
        g.addEdge("c", "a");
    }

    @Test
    public void testNodeSetSize() {
        assertEquals(5, g.nodeSet().size());
    }

    @Test
    public void testNodeSetElements() {
        assertThat(g.nodeSet(), hasItems("a", "b", "c", "d", "e"));
    }
```

Variante migliorativa (2/2)

```java

    @Test
    public void testLinkedNodes() {
        assertEquals(2, g.linkedNodes("c").size());
        assertThat(g.linkedNodes("c"), hasItems("a", "d"));
    }

    @Test
    public void testGetPath() {
        assertEquals(Arrays.asList("b", "c", "a"), g.getPath("b", "a"));
    }
}
```